

B.TECH CSE · OOPS PROGRAMMING LAB

Secure Access Manager

with Intelligent Role-Based Access Control

Vanshika Rawat · 590013163

Saksham Soni · 590012301

Shreya Uniyal · 590014941

Java · Swing · JDBC · MySQL · SHA-256



Introduction — Why This System?



Basic Login is Insecure

Username + password only — no tracking, no behavior analysis



No Brute-Force Protection

Repeated wrong attempts go unnoticed in standard systems



No Role-Based Control

All users get the same access regardless of their role.



Our Solution

Track attempts, hash passwords, account blocking after multiple attempts, control access by role

Objectives

01 Secure Authentication

SHA-256 hash via `HashUtil.sha256()` so that the password is never stored in plain text

02 Role-Based Access Control

`canAccessFeature()` in `Admin/Student` decides who sees what in `DashboardUI`

03 Account Security and Attack Prevention

`failed_attempts` tracked in DB; account blocked after repeated login failures

04 Admin Monitoring

`getAllUsers()` in `UserDAO` feeds the Admin Panel table and all users visible in one view

2.1 Functional Requirements

The system must satisfy the following functional requirements:

- Allow users to log in using a username and password.
- Encrypt all passwords using SHA-256 before storing them in the database.
- Track the number of failed login attempts per user.
- Temporarily block a user account for two minutes after three consecutive failed attempts.
- Maintain a history of failed login attempts without resetting them.
- Display an Admin Panel accessible only to users with the ADMIN role.
- Allow the Admin Panel to view all registered users and their account status.
- Restrict Student users to the Dashboard and Profile sections only.
- Run a background daemon thread that resets failed attempt counters every two minutes.

2.2 Non-Functional Requirements

- Security: Passwords must never be stored or transmitted in plain text.
- Usability: The graphical interface must be clean and intuitive for end users.
- Reliability: Database errors must be caught and reported gracefully without crashing the application.
- Maintainability: Code must follow a layered architecture separating UI, business logic, and data access.

2.3 Software Requirements

Component	Details
Programming Language	Java JDK 17 or above
IDE	IntelliJ IDEA / Eclipse / VS Code
Database	MySQL 8.0
Database Driver	MySQL Connector/J (JDBC Driver)
GUI Framework	Java Swing
Version Control	Git / GitHub

System Overview - Login Flow

1



**User Enters
Credentials**
LoginUI.java

3



**Login Validation &
Security Checked**
LoginService.java

2

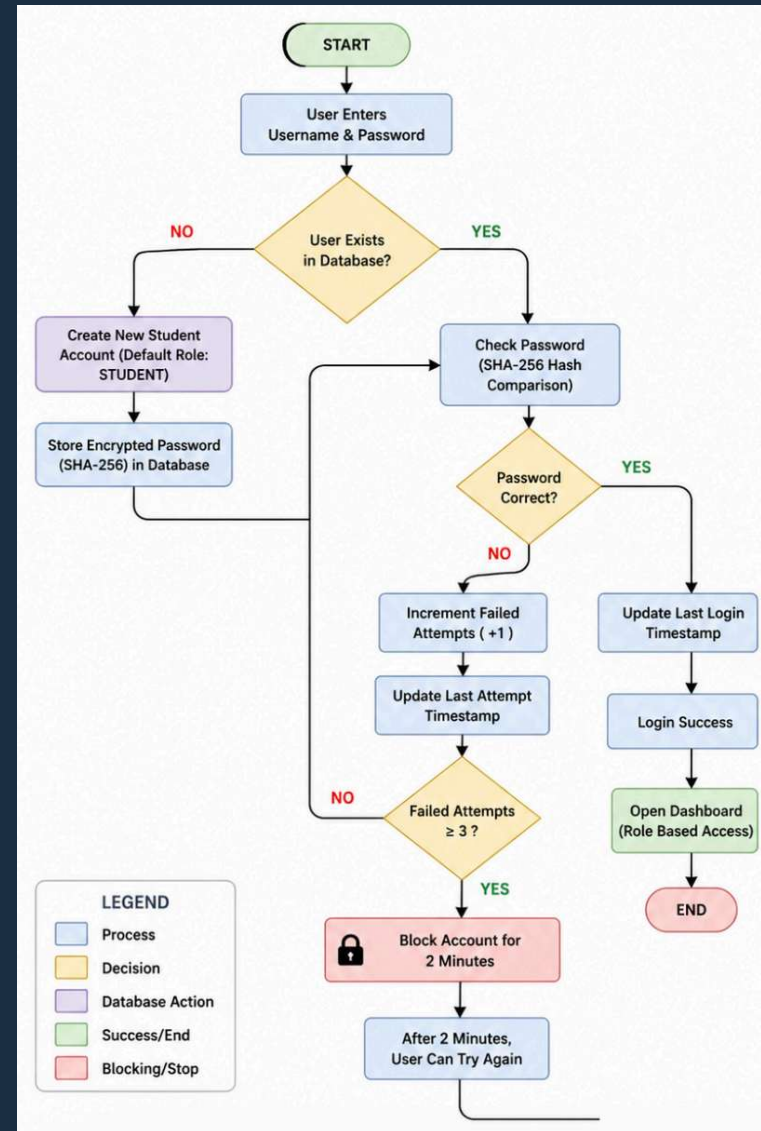


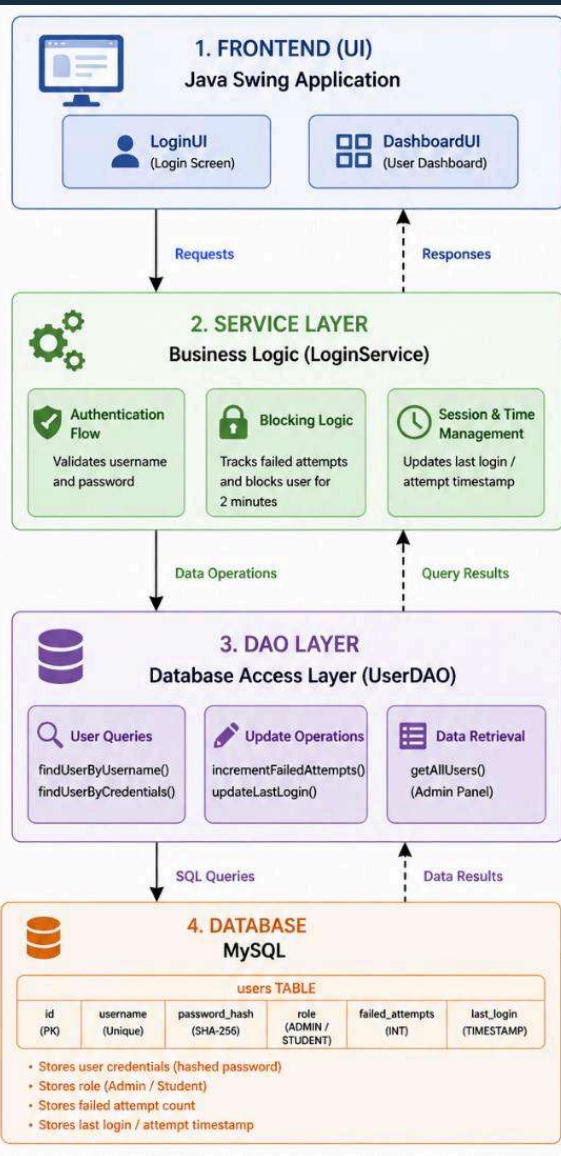
**handleLogin()
Runs**
Button click event

4



**Role-Based
Access Granted**
DashboardUI.java





MANAGES

- User input and interaction
- Display login results and messages
- Show dashboard based on role
- Sends requests to Service Layer

MANAGES

- Core business rules
- Validates credentials
- Increments failed attempts
- Applies temporary blocking
- Updates timestamps
- Decides access (Admin / Student)

MANAGES

- All SQL queries
- Insert / Update / Select operations
- Communicates with MySQL database
- Returns data to Service Layer

MANAGES

- Persistent data storage
- Securely stores all user information
- Provides data to the application when requested

Java Features Where Used in Code

JDBC

UserDAO.java

PreparedStatement queries MySQL
findUserByCredentials(), getAllUsers()

Exception Handling

LoginService.java

Contains Logics, Blocking, Timestamps

LoginService talks to UserDAO for data, UserDAO talks to MySQL via DBConnection, HashUtil keeps passwords safe, SecurityAnalyzer runs alongside everything on a daemon thread, and the model classes (User, Admin, Student) carry the OOP structure that makes role-based behaviour work through polymorphism

Swing UI

LoginUI.java

JFrame + JTextField + JPasswordField
handleLogin() on button ActionListener

SHA-256 Hashing

HashUtil.java

MessageDigest.getInstance("SHA-256")
converts password to 64-char hex

```
Connection conn = DriverManager.getConnection(
    "jdbc:mysql://localhost:3306/secure_login_db",
    "root",
    "password"
);
```

```
try {
    Connection conn = DBConnection.getConnection();
} catch (SQLException e) {
    e.printStackTrace();
}
```

```
JFrame frame = new JFrame("Login");
frame.setSize(300, 200);
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
frame.setVisible(true);
```

```
MessageDigest md = MessageDigest.getInstance("SHA-256");
byte[] hash = md.digest(password.getBytes());

StringBuilder hex = new StringBuilder();
for (byte b : hash) {
    hex.append(String.format("%02x", b));
}
```

Classes Used — With Purpose

User.java

Abstract base class — username, role, canAccessFeature()

Admin.java

Extends User — canAccessFeature() returns true for everything

Student.java

Extends User — canAccessFeature() allows only dashboard + profile

LoginService.java

login() method — checks block, hashes password, validates, blocking logic

UserDAO.java

DB queries — findUserByCredentials(), incrementFailedAttempts(), etc.

LoginUI.java

Swing window — handleLogin() runs when Login button is clicked

DashboardUI.java

Role-based tabs — Admin tab shown only if canAccessFeature() is true

☞ During demo: open each .java file and point to the class declaration — explain in 1 line

OOP Concepts — With Code

Abstraction

User.java is abstract — cannot be instantiated directly

Inheritance

Admin extends User | Student extends User

Polymorphism

canAccessFeature() behaves differently per subclass

Encapsulation

All User fields private — accessed via getters only

User.java — abstraction

```
public abstract class User {  
    public abstract boolean  
        canAccessFeature(String f);  
}
```

Admin.java — inheritance + polymorphism

```
public class Admin extends User {  
    public boolean canAccessFeature  
        (String f) { return true; }  
}
```

Student.java — restricted access

```
public class Student extends User {  
    public boolean canAccessFeature(String f) {  
        return f.equals("dashboard")  
            || f.equals("profile");  
    }  
}
```

2. Inheritance

What it is: A subclass acquires the fields and methods of its parent class.

Where used: Admin.java and Student.java both extend User

```
public class Admin extends User {
    public Admin(int id, String username, int failedAttempts) {
        super(id, username, "ADMIN", failedAttempts);
    }
}

public class Student extends User {
    public Student(int id, String username, int failedAttempts) {
        super(id, username, "STUDENT", failedAttempts);
    }
}
```

4. Encapsulation

What it is: Fields are kept private. Outside classes can only read or write them through controlled getter/setter methods.

Where used: User.java

```
public abstract class User {
    private int id;
    private String username;
    private String role;
    private int failedAttempts;
    private Timestamp lastLogin;

    public int getId() { return id; }
    public String getUsername() { return username; }
    public String getRole() { return role; }
    public int getFailedAttempts() { return failedAttempts; }
    public Timestamp getLastLogin() { return lastLogin; }
    public void setLastLogin(Timestamp t) { this.lastLogin = t; }
}
```

No class can do `user.failedAttempts = -999`. They can only read through `getFailedAttempts()`. The only setter exposed is `setLastLogin()` — because that's the only field that legitimately needs to be set after construction. Everything else is read-only from outside.

3. Polymorphism

What it is: The same method call behaves differently depending on the actual object type at runtime.

Where used: `canAccessFeature()` — called the same way, different result per role

```
// Admin.java
public boolean canAccessFeature(String feature) {
    return true; // Admin can access everything
}

// Student.java
public boolean canAccessFeature(String feature) {
    return feature.equals("dashboard") || feature.equals("profile");
}

// DashboardUI.java — same call, different outcome
if (user.canAccessFeature("admin_panel")) {
    tabs.addTab("Admin Panel", buildAdminTab());
}
```

[user](#) here is of type `User` — the abstract parent. At runtime, if it's an `Admin` object, the `Admin` version runs and returns `true`. If it's a `Student`, the `Student` version runs and returns `false`. `DashboardUI` doesn't know or care which one — it just calls the method.

4. Encapsulation

What it is: Fields are kept private. Outside classes can only read or write them through controlled getter/setter methods.

Where used: User.java

```
public abstract class User {
    private int id;
    private String username;
    private String role;
    private int failedAttempts;
    private Timestamp lastLogin;

    public int getId() { return id; }
    public String getUsername() { return username; }
    public String getRole() { return role; }
    public int getFailedAttempts() { return failedAttempts; }
    public Timestamp getLastLogin() { return lastLogin; }
    public void setLastLogin(Timestamp t) { this.lastLogin = t; }
}
```

No class can do `user.failedAttempts = -999`. They can only read through `getFailedAttempts()`. The only setter exposed is `setLastLogin()` — because that's the only field that legitimately needs to be set after construction. Everything else is read-only from outside.

1. Abstract Class

What it is: A class that cannot be instantiated directly. It defines a common structure and forces subclasses to implement certain methods.

Where used: User.java

```
public abstract class User {  
    private int id;  
    private String username;  
    private String role;  
    private int failedAttempts;  
  
    public abstract boolean canAccessFeature(String feature);  
  
    public int getTrustScore() {  
        return Math.max(0, 100 - (failedAttempts * 20));  
    }  
}
```

User cannot be created with **new User()**. You can only create **new Admin()** or **new Student()**. The method `canAccessFeature` is abstract; it has no body here. Every subclass is forced to write its own version.

2. Generics

What it is: Writing code that works with any type while keeping type safety. Defined using `<T>` or a specific type in `<>`.

Where used: UserDao.java (`List<User>`)

```
public List<User> getAllUsers() throws SQLException {  
    List<User> users = new ArrayList<>();  
  
    while (rs.next()) {  
        users.add(buildUser(rs));  
    }  
    return users;  
}
```

List<User> means this list can **only hold User objects** and the compiler will **refuse** to let you add a **String** or an **Integer** into it. Without generics it would be a raw List, and you'd need to cast every element manually with no compile-time safety. `ArrayList<>` on the right side uses the diamond operator which Java infers the type from the left side.

7. Lambda Expression

What it is: A short anonymous function written inline. Syntax: (parameters) -> { body }. Used wherever a functional interface is expected.

Where used: LoginUI.java (button click handler and Timer)

```
loginButton.addActionListener(e -> handleLogin());
```

```
Timer timer = new Timer(600, e -> {  
    dispose();  
    new DashboardUI(user).setVisible(true);  
});
```

addActionListener expects an ActionListener — a functional interface with one method actionPerformed(ActionEvent e). Instead of writing a whole anonymous class, the lambda e -> handleLogin() is shorthand for:

```
loginButton.addActionListener(new ActionListener() {  
    public void actionPerformed(ActionEvent e) {  
        handleLogin();  
    }  
});
```

Same result, four lines less. e is the ActionEvent parameter we don't use it here, but it must be declared.

11. Method Overriding

What it is: A subclass provides its own implementation of a method already defined in the parent class. Marked with @Override.

Where used: Admin.java, Student.java, DashboardUI.java

```
// Student.java  
@Override  
public boolean canAccessFeature(String feature) {  
    return feature.equals("dashboard") || feature.equals("profile");  
}
```

```
// DashboardUI.java anonymous class  
@Override  
public boolean isCellEditable(int row, int col) {  
    return false;  
}
```

@Override tells the compiler "I intend to override a parent method". If you misspell the method name, the compiler throws an error instead of silently creating a brand new method that never gets called.

Security Features — Code Flow

Password Hashing

```
HashUtil.sha256(password)
```

Track Failures

```
userDAO.incrementFailedAttempts()
```

Block After 3

```
if (attempts >= 3) → BLOCKED
```

login() — complete decision flow
// LoginService.java → login()

1. Null check → if empty → ERROR
2. Find user by username
if attempts >= 3 → BLOCKED
3. Hash input password via SHA-256
Compare with DB hash
if no match → increment attempts
→ INVALID_CREDENTIALS
4. If correct -> reset attempts
5. Reset attempts → return SUCCESS
Open DashboardUI

UI Demo — Live Walkthrough

Login Screen

Secure Login System

 **Secure Login**

Username:

Password:

Login

Dashboard

Dashboard — admin (ADMIN)

Welcome, admin! Role: ADMIN

Dashboard **Admin Panel**

Username : admin
Role : ADMIN
Failed Attempts: 0

Your account is active.

Logout

Admin Panel

Dashboard — admin (ADMIN)

Welcome, admin! Role: ADMIN

Dashboard **Admin Panel**

All Registered Users

	Username	Role	Failed Attempts	Last Login
	admin	ADMIN	0	2026-04-28
	student1	STUDENT	3	2026-04-28
	student2	STUDENT	3	2026-04-28

Delete Selected Users

Logout

➡ 1. Type admin / admin123 → click Login → handleLogin() runs → opens DashboardUI

➡ 2. Show Admin Panel tab → getAllUsers() queries DB → table shows failed attempts and user roles

➡ 3. Type wrong password 3× → failed_attempts increments → account blocked message



Valid login → success

User successfully logged in and redirected to Dashboard.



Wrong password → failed_attempts increases

Incorrect password increments failed_attempts in database.



3 wrong attempts → account blocked

After 3 failed attempts, account gets blocked and further login is denied.



Admin login → access to Admin Panel

Admin can log in and access the Admin Panel with all features.



Last login timestamp updates on successful login

last_login timestamp is updated in database on every successful login.

Admin Panel - User Management

ID	Username	Role	Failed Attempts	Last Login

Welcome, admin!

Role: ADMIN

Dashboard

Admin Panel

All Registered Users

ID	Username	Role	Failed Attempts	Last Login
1	admin	ADMIN	0	2026-04-28 22...
2	student1	STUDENT	3	2026-04-28 10...
3	student2	STUDENT	3	2026-04-28 09...

Delete Selected User



Conclusion:

- Secure authentication using SHA-256 hashing
- Tracks failed login attempts and prevents brute-force attacks
- Role-based access control implemented
- Admin panel allows monitoring of user activity
- Last login timestamp improves tracking



Future Scope:

- OTP / Two-factor authentication
- Email alerts for blocked accounts
- Password reset feature



Requirements vs Implementation:

Requirement	Implemented
Password hashing	✓
Failed attempts tracking	✓
Account blocking	✓
Role-based access	✓
Admin monitoring	✓
Last login timestamp	✓